

■ 下列文件物理结构中，适合随机访问且易于文件扩展的是

- A .连续结构
- B.索引结构
- C.链式结构且磁盘块定长
- D.链式结构且磁盘块变长

■ 某文件系统空间的最大容量为4TB ($1\text{TB}=2^{40}$), 以磁盘块为基本分配单位。磁盘块大小为1KB。文件控制块(FCB)包含一个512B的索引表区。请回答下列问题。

■ (1)假设索引表区仅采用直接索引结构, 索引表区存放文件占用的磁盘块号, 索引表项中块号最少占多少字节? 可支持的单个文件最大长度是多少字节?

■ (2)假设索引表区采用如下结构:第0~7字节采用<起始块号, 块数>格式表示文件创建时预分配的连续存储空间, 其中起始块号占6B, 块数占2B;剩余504字节采用直接索引结构, 一个索引项6B, 则可支持的单个文件最大长度是多少字节?为了使单个文件的长度达到最大, 请指出起始块号和块数分别所占字节数的合理值并说明理由

○

- (1)文件系统中所能容纳的磁盘块总数为 $4\text{TB}/1\text{KB}=2^{32}$ 。要完全表示所有磁盘块，索引项中的块号最少要占 $32/8=4\text{B}$ 。
- 而索引表区仅采用直接索引结构，故 512B 的索引表区能容纳 $512\text{B}/4\text{B}=2^7$ 个索引项。每个索引项对应一个磁盘块，所以该系统可支持的单个文件最大长度是 $128 \times 1\text{KB}=128\text{KB}$ 。

- (2)块号占6B、块数占2B时，共可以表示 2^{16} 个磁盘块，即 $2^{26}=64\text{MB}$ ；直接索引区共 $504\text{B}/6\text{B}=84$ 个索引项。所以该系统可支持的单个文件最大长度是 $2^{26}\text{B}+84\text{KB}=65620\text{KB}$ 。
- 合理的起始块号和块数所占字节数分别为4,4(或1,7或2,6或3,5)。
- 理由:块数占4B或以上，就可表示4TB大小的文件长度，达到文件系统的空间上限。

- A file system uses 256-byte physical blocks. Each file has a directory entry giving the file name, location of the first block, length of file, and last block position. Assuming the last physical block read is 100, block 100 and the directory entry are already in main memory.
- For the following two file allocation algorithms, how many physical blocks must be read to access the specific block 600 (including the reading of block 600 itself), and why?
- (1) contiguous allocation
- (2) linked allocation

■ (1) 1

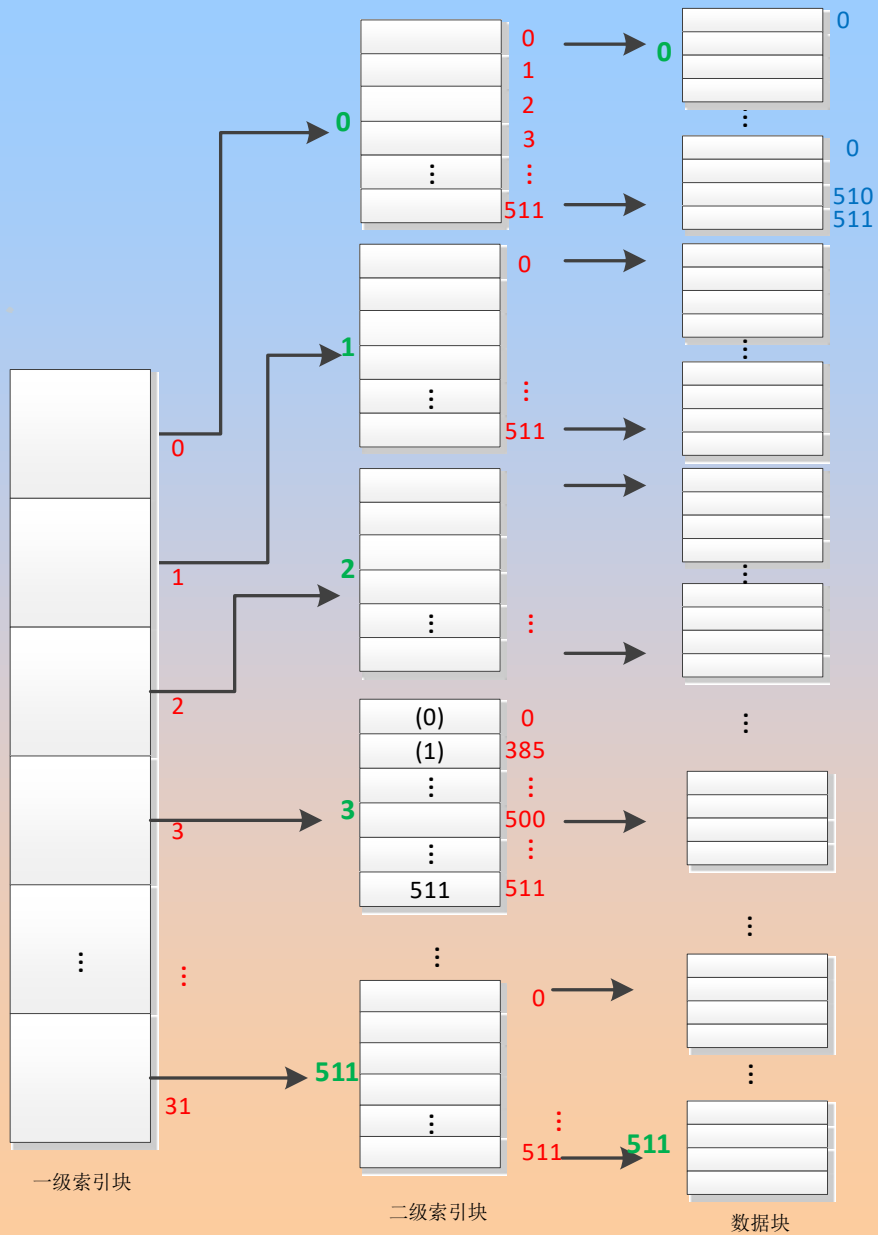
Contiguous allocation supports direct access on arbitrary physical blocks in a file. To access physical block 600, the block 600 can be directly read into memory.

■ (2) 500

Linked allocation is used only for sequential-access files, file system can only read blocks one by one, directed by file pointers.

To find the physical block 600 in the file, file system locates on the 101th block following the 100th block just read, and reads block 101, block 102, ..., until block 600.

- Consider a file system in which a directory entry can store up to 32 disk block addresses. For the files that is not larger than 32 blocks, the 32 addresses in the entry serves as the file's index table.
- For the files that is larger than 32 blocks, each of the 32 addresses points to an indirect block that in turn points to 512 file blocks on the disk, and the size of a block is 512-bytes.
- What is the largest size of a file?

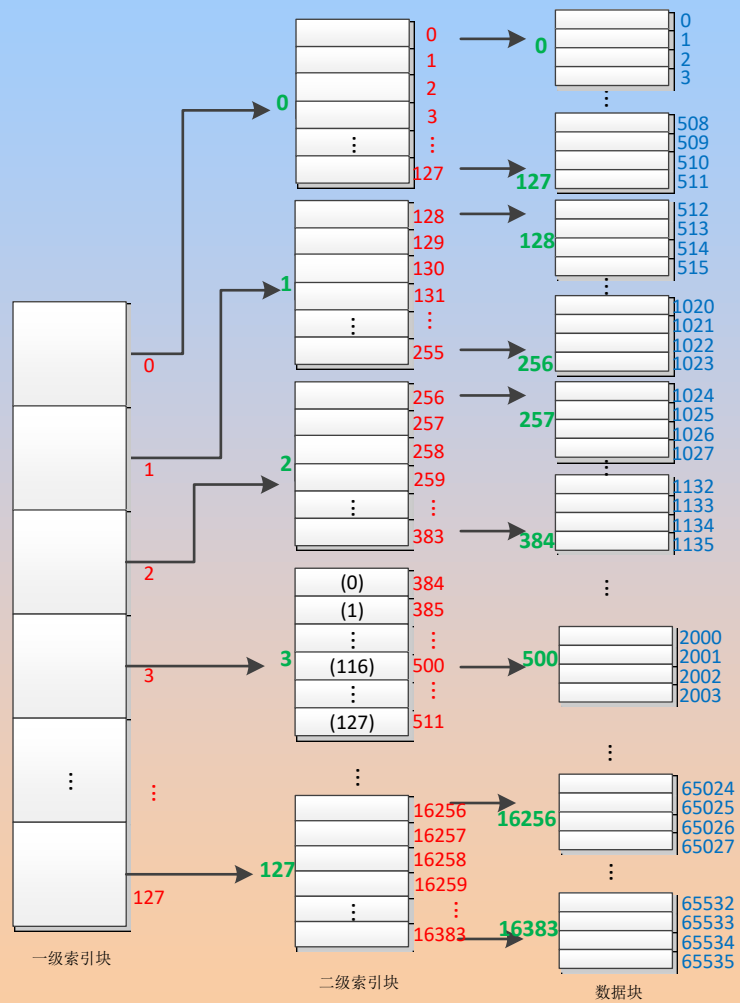


- $32 \times 512 = 16384$ file blocks
 $16384 \times 512 = 8388608$ bytes

or

- $2^5 \times 2^9 \times 2^9 = 2^{23}$ bytes

- In the file system on a disk with physical block sizes of 512 bytes, a file is made up of 128-byte logical records, and each logical record cannot be separately stored in two different blocks. The disk space of the file is organized on the basis of indexed allocation, and a block address is stored in 4 bytes. Suppose that 2-level index blocks be used to manage the data blocks of the file, answer the following questions:
 - 1) What is the largest size of the file?
 - 2) Given 2000, the number of a logical record in the file, how to find out the physical address of the record 2000 in accordance with the 2-level index blocks.



- 图说明：图中有三种块，一级索引块、二级索引块和数据块（物理块），二级索引块和数据块的块数用绿色数值标注。一级索引块和二级索引块的右边红色数值是索引表项，代表了逻辑块指针的编号，数据块右边蓝色的数值是记录号，代表了逻辑记录单元的编号，编号从0开始，连续记录，由于空间限制，只标注了部分记录号。
- 图中的一级索引块、每个二级索引块和每个数据块大小均为**512B**，由于空间限制，块的大小画的不一样大。
- 因为块地址存储在**4**字节中，因此一级索引块共有 $512\text{B}/4\text{B}=128$ 个指针，每个指针指向一个二级索引块，因此二级索引中共有**128**个二级索引块，每个二级索引块也有**128**个指针，每个指针指向一个数据块，又因为文件是由**128B**的逻辑记录单元组成，因此每个数据块有**4**个**128B**的逻辑记录单元。

■ 问题（1）支持的文件最大尺寸为：

$$128 * 128 * 512 \text{B} = 8192 \text{KB}$$

■ 问题（2）第2000逻辑记录的位置在：

$$2000 / 4 \text{个} = 500$$

$$500 / 128 = 3 \text{余} 116$$

第2000个逻辑记录在一级索引块的第3块上和二级索引块的116块上，因为记录号从0开始，在图上显式是一级索引的3号指针指向二级索引的500号指针，二级索引的第500号指针是第四个二级索引块中116号指针。

■ 1)

$$512/4=128$$

$$128*128*512=128*64 \text{ KB}=8192\text{KB}$$

$$\text{or } =2^{23} \text{ bytes} = 8 \text{ MB}$$

$$\text{or } = 8388680$$

■ 2)

$$512/128=4$$

$$2000/4=500$$

in 3 block in the first –level index block

in 116 block in the second-level index block

- Consider a paging system with the page table stored in memory.
- a. If a memory reference takes 300 time unit, how long does it take to access an instruction or data in a page that has been paged into memory?
- b. If we add TLB (*translation look-aside buffers*), and 80 percent of all page-table entries can be found in the TLB, what is the effective memory access time?
- (Assume that finding a page-table entry in the associative registers takes 20 time unit, if the entry is there.)

- $300 \times 2 = 600$

- $0.8 \times (300 + 20) + 0.2 \times (600 + 20) = 256 + 124 = 380$